

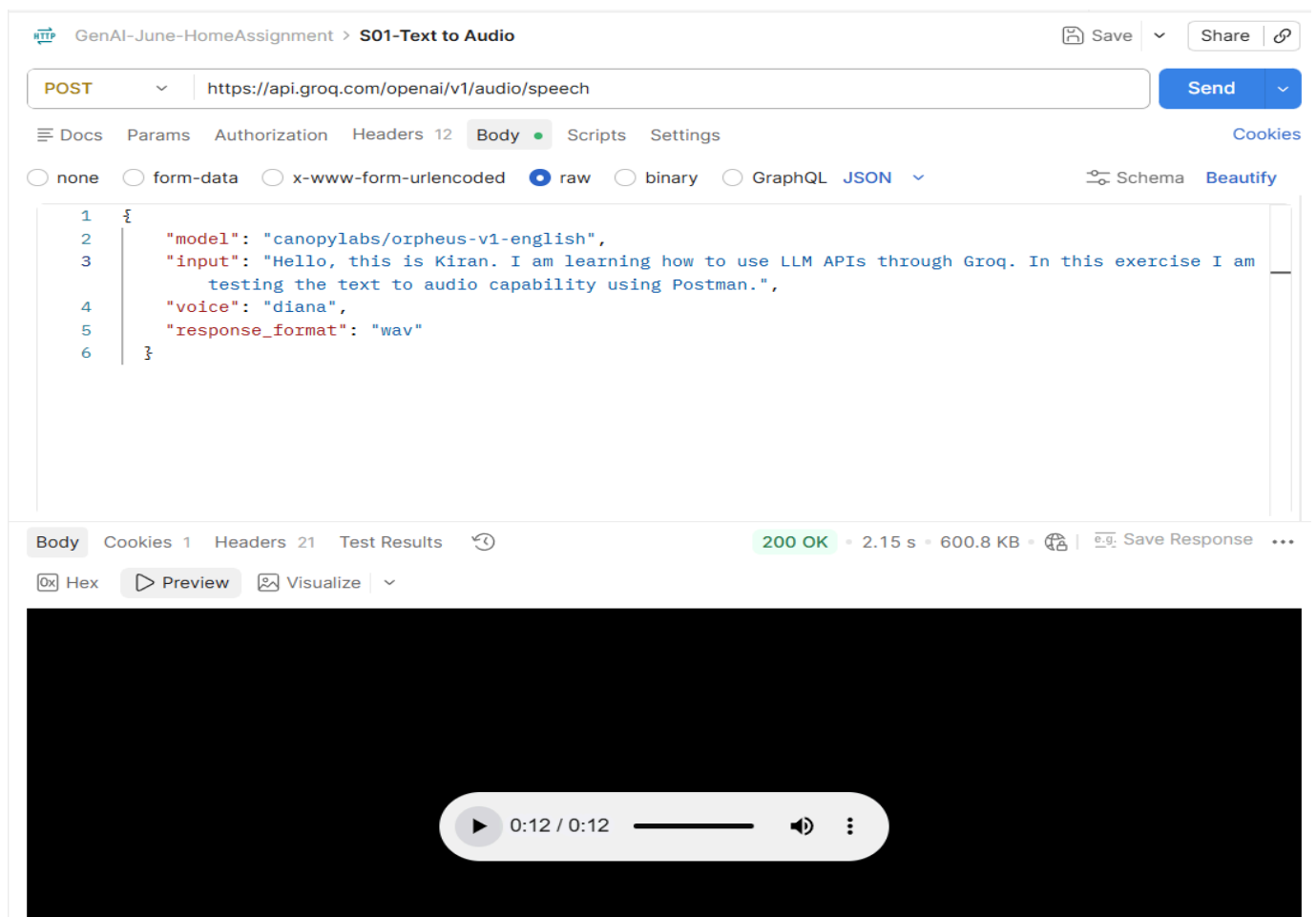
LLM API Integration

S01 – Text to Audio

Request Body:

```
{  
  "model": "canopylabs/orpheus-v1-english",  
  "input": "Hello, this is Kiran. I am learning how to use LLM APIs through Groq. In  
this exercise I am testing the text to audio capability using Postman.",  
  "voice": "diana",  
  "response_format": "wav"  
}
```

Postman Screenshot:



S02 – Text to Audio

Request Body(Malayalam):

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "ഹലോ, ഞാൻ LLM API പഠിക്കുകയാണ്. LLM API എന്താണെന്ന് ലളിതമായി വിശദീകരിക്കാമോ?"
    }
  ]
}
```

Response Body:

```
{
  "id": "chatcmpl-b3c0c646-92d9-4e47-9cfe-c43587e5dad9",
  "object": "chat.completion",
  "created": 1781711293,
  "model": "llama-3.3-70b-versatile",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "ഹലോ, സുഹൃത്ത്, നിങ്ങളുടെ ചോദ്യത്തിന് ഉത്തരം നൽകാൻ സന്തോഷമാണ്.\n\nLLM API എന്ന് Large Language Model API എന്ന് നമുക്ക് വിളിക്കാം. ഇത് ഒരു പരമ്പരാഗതമായ ഇൻ്റർഫേസ് ആണ്, ഇത് വലിയ ഭാഷാ മോഡലുകൾ ഉപയോഗിച്ച് വാചകം വിശകലനം ചെയ്യാനും ഉത്തരം നൽകാനും സാധിക്കും.\n\nലളിതമായി പറഞ്ഞാൽ, LLM API ഒരു കമ്പ്യൂട്ടർ പരമ്പരാഗതമാണ്, അത് മനുഷ്യരുടെ ഭാഷാ ശൈലി പഠിക്കുകയും വാചകം വിശകലനം ചെയ്യുകയും ഉത്തരം നൽകുകയും ചെയ്യുന്നു. ഈ API ഉപയോഗിച്ച്, നിങ്ങൾക്ക് വാചകം ഉൽപ്പാദിപ്പിക്കാനും വാചകം വിശകലനം ചെയ്യാനും സാധിക്കും.\n\nഉദാഹരണത്തിന്, നിങ്ങൾക്ക് ഒരു ചാറ്റ്ബോട്ട് സൃഷ്ടിക്കാൻ ആഗ്രഹമുണ്ടെങ്കിൽ, നിങ്ങൾക്ക് LLM API ഉപയോഗിച്ച് അത് സൃഷ്ടിക്കാം. നിങ്ങൾക്ക് ഒരു വാചകം എഴുതാൻ ആഗ്രഹമുണ്ടെങ്കിൽ, നിങ്ങൾക്ക് LLM API
```

ഉപയോഗിച്ച് അത് എഴുതാം.\n\nഇങ്ങനെ, LLM API ഒരു ശക്തമായ ഉപകരണമാണ്, അത് നിങ്ങളുടെ വാചകം വിശകലനം ചെയ്യാനും ഉത്തരം നൽകാനും സാധിക്കും."

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.161207237,
  "prompt_tokens": 157,
  "prompt_time": 0.015339893,
  "completion_tokens": 1204,
  "completion_time": 3.431508051,
  "total_tokens": 1361,
  "total_time": 3.446847944
},
"usage_breakdown": null,
"system_fingerprint": "fp_45180df409",
"x_groq": {
  "id": "req_01kvb49fkjf6saslyvjwt3708ky",
  "seed": 59025072
},
"service_tier": "on_demand"
}
```

JSON to Plain Text output:

ഹലോ, സുഹൃത്തേ, നിങ്ങളുടെ ചോദ്യത്തിന് ഉത്തരം നൽകാൻ സന്തോഷമാണ്.

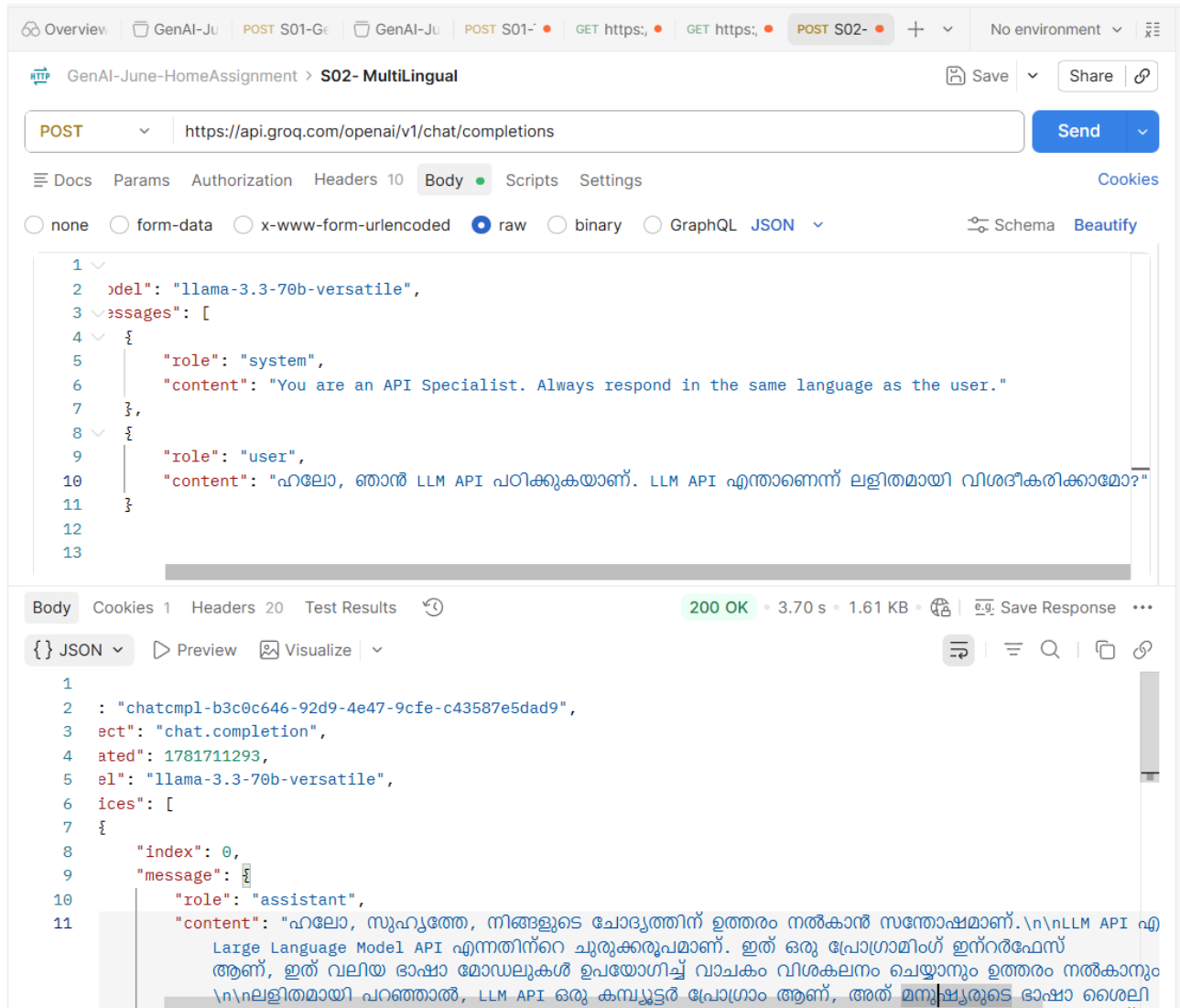
LLM API എന്നത് Large Language Model API എന്നതിന്റെ ചുരുക്കരൂപമാണ്. ഇത് ഒരു പ്രോഗ്രാമിംഗ് ഇന്റർഫേസ് ആണ്, ഇത് വലിയ ഭാഷാ മോഡലുകൾ ഉപയോഗിച്ച് വാചകം വിശകലനം ചെയ്യാനും ഉത്തരം നൽകാനും സാധിക്കും.

ലളിതമായി പറഞ്ഞാൽ, LLM API ഒരു കമ്പ്യൂട്ടർ പ്രോഗ്രാം ആണ്, അത് മനുഷ്യരുടെ ഭാഷാ ശൈലി പഠിക്കുകയും വാചകം വിശകലനം ചെയ്യുകയും ഉത്തരം നൽകുകയും ചെയ്യുന്നു. ഈ API ഉപയോഗിച്ച്, നിങ്ങൾക്ക് വാചകം ഉൽപ്പാദിപ്പിക്കാനും വാചകം വിശകലനം ചെയ്യാനും സാധിക്കും.

ഉദാഹരണത്തിന്, നിങ്ങൾക്ക് ഒരു ചാറ്റ്ബോട്ട് സൃഷ്ടിക്കാൻ ആഗ്രഹമുണ്ടെങ്കിൽ, നിങ്ങൾക്ക് LLM API ഉപയോഗിച്ച് അത് സൃഷ്ടിക്കാം. നിങ്ങൾക്ക് ഒരു വാചകം എഴുതാൻ ആഗ്രഹമുണ്ടെങ്കിൽ, നിങ്ങൾക്ക് LLM API ഉപയോഗിച്ച് അത് എഴുതാം.

ഇങ്ങനെ, LLM API ഒരു ശക്തമായ ഉപകരണമാണ്, അത് നിങ്ങളെ വാചകം വിശകലനം ചെയ്യാനും ഉത്തരം നൽകാനും സാധിക്കും.

Postman Screenshot:



S03-GenerateAPITestcasesFromSwaggerCode

Request Body:

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
```

```

{
  "role": "system",
  "content": "You are a Senior API Test Automation Architect with strong expertise
in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
},
{
  "role": "user",
  "content": "I - INSTRUCTION\n\nGenerate Rest Assured Java code for a Login
API.\n\nMANDATORY RULES:\n\n1. Generate ONLY Java Rest Assured code.\n2. Do NOT
include explanations, markdown, or extra text.\n3. Use TestNG as the test runner.\n4.
Use meaningful class, method, and variable names.\n5. Include required imports.\n6.
Use reusable base URI setup.\n7. Add assertions for status code and important response
fields.\n8. Keep the code clean and executable.\n\nC - CONTEXT\n\nThe code will be
used by QA engineers for API automation practice.\n\nE - EXAMPLE\n\nLogin API
operations:\n\n* Login with valid credentials\n* Login with invalid credentials\n*
Logout\n\nP - PERSONA\n\nYou are a Senior API Test Automation Architect creating
reusable automation code for QA teams.\n\nO - OUTPUT\n\nReturn ONLY executable Java
code with:\n\n* One test class\n* @BeforeClass setup\n* Test methods for validLogin\n*
Test methods for invalidLogin\n* Test methods for logout\n* Request body creation\n*
Response validation\n\nT - TONE\n\nProfessional and production-ready."
}
]
}

```

Response Body:

```

{
  "id": "chatcmpl-f1222cde-38de-414a-b56c-d8207f6f3949",
  "object": "chat.completion",
  "created": 1781712297,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport
io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport
org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport
static io.restassured.RestAssured.*;\nimport static org.hamcrest.Matchers.*;\n\npublic
class LoginApiTest {\n\n    private static final String BASE_URI =
\n\"https://api.example.com\";\n    private static String authToken;\n\n

```

```

@BeforeClass\n    public void setUp() {\n        RestAssured.baseURI = BASE_URI;\n    }\n\n    @Test\n    public void validLogin_ShouldReturn200AndToken() {\n        String\n        requestBody = \"{ \\\"username\\\": \\\"validUser\\\", \\\"password\\\":\n        \\\"validPass\\\" }\";\n        Response response = given()\n        .contentType(ContentType.JSON)\n        .body(requestBody)\n        .when()\n        .post(\"/login\")\n        .then()\n        .assertThat()\n        .statusCode(200)\n        .body(\"token\", notNullValue())\n        .extract()\n        .response();\n        authToken =\n        response.jsonPath().getString(\"token\");\n\n        @Test(dependsOnMethods =\n        \"validLogin_ShouldReturn200AndToken\")\n        public void\n        logout_ShouldInvalidateToken() {\n            given()\n            .header(\"Authorization\", \"Bearer \" + authToken)\n            .when()\n            .post(\"/logout\")\n            .then()\n            .assertThat()\n            .statusCode(200)\n            .body(\"message\", equalTo(\"Logged out\n            successfully\"));\n\n            @Test\n            public void invalidLogin_ShouldReturn401()\n            {\n                String requestBody = \"{ \\\"username\\\": \\\"invalidUser\\\", \\\"password\\\": \\\"wrongPass\\\" }\";\n                given()\n                .contentType(ContentType.JSON)\n                .body(requestBody)\n                .when()\n                .post(\"/login\")\n                .then()\n                .assertThat()\n                .statusCode(401)\n                .body(\"error\", equalTo(\"Invalid credentials\"));\n            }\n        }\n    }\n}

```

"reasoning": "We need to output Java code only, no markdown, no explanations. Should include imports, class, methods, beforeclass. Use RestAssured, TestNG, Maven style. Reusable base URI. Include request body creation (maybe using a Map or JSONObject). Include assertions for status code and important fields (e.g., token, message). Provide three test methods.\n\nMake sure code compiles. Use static imports for RestAssured.given and Matchers.* maybe. Provide base URI in @BeforeClass. Also maybe store token for logout.\n\nLet's produce code:\n\nimport io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport static io.restassured.RestAssured.*;\nimport static org.hamcrest.Matchers.*;\n\npublic class LoginApiTest {\n private static final String BASE_URI = \"https://api.example.com\";\n private static String authToken;\n\n @BeforeClass\n public void setUp() {\n RestAssured.baseURI = BASE_URI;\n }\n\n @Test\n public void validLogin_ShouldReturn200AndToken()\n {\n String requestBody = \"{ \\\"username\\\": \\\"validUser\\\", \\\"password\\\": \\\"validPass\\\" }\";\n Response response =\n given()\n .contentType(ContentType.JSON)\n .body(requestBody)\n .when()\n .post(\"/login\")\n .then()\n .assertThat()\n .statusCode(200)\n .body(\"token\", notNullValue())\n .body(\"userId\", notNullValue())\n }\n\n @Test\n public void logout_ShouldInvalidateToken()\n {\n given()\n .header(\"Authorization\", \"Bearer \" + authToken)\n .when()\n .post(\"/logout\")\n .then()\n .assertThat()\n .statusCode(200)\n .body(\"message\", equalTo(\"Logged out successfully\"));\n }\n\n @Test\n public void invalidLogin_ShouldReturn401()\n {\n String requestBody = \"{ \\\"username\\\": \\\"invalidUser\\\", \\\"password\\\": \\\"wrongPass\\\" }\";\n given()\n .contentType(ContentType.JSON)\n .body(requestBody)\n .when()\n .post(\"/login\")\n .then()\n .assertThat()\n .statusCode(401)\n .body(\"error\", equalTo(\"Invalid credentials\"));\n }\n}\n

```

.extract()\n                .response();\n\n                authToken =
response.jsonPath().getString(\"token\");\n                }\n\n                @Test(dependsOnMethods =
\"validLogin_ShouldReturn200AndToken\")\n                public void
logout_ShouldInvalidateToken() {\n                given()\n                .header(\"Authorization\", \"Bearer \" + authToken)\n                .when()\n                .post(\"/logout\")\n                .then()\n                .assertThat()\n                .statusCode(200)\n                .body(\"message\", equalTo(\"Logged out
successfully\"));\n                }\n\n                @Test\n                public void invalidLogin_ShouldReturn401()
{\n                String requestBody = \"{ \\\"username\\\": \\\"invalidUser\\\",
\\\"password\\\": \\\"wrongPass\\\" }\";\n                given()\n                .contentType(ContentType.JSON)\n                .body(requestBody)\n                .when()\n                .post(\"/login\")\n                .then()\n                .assertThat()\n                .statusCode(401)\n                .body(\"error\", equalTo(\"Invalid credentials\"));\n                }\n\n                }\n\n                Make sure imports include json etc. That's fine. Provide code only."

```

```

        },
        "logprobs": null,
        "finish_reason": "stop"
    }
],
"usage": {
    "queue_time": 0.32055666,
    "prompt_tokens": 329,
    "prompt_time": 0.01943877,
    "completion_tokens": 963,
    "completion_time": 2.024404822,
    "total_tokens": 1292,
    "total_time": 2.043843592,
    "completion_tokens_details": {
        "reasoning_tokens": 542
    }
},
"usage_breakdown": null,
"system_fingerprint": "fp_e5b4e54fbb",
"x_groq": {
    "id": "req_01kvb585d0ej7931n37y4dygc4",
    "seed": 1141959414
},
"service_tier": "on_demand"
}

```

JSON to Plain Text output:

```
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

public class LoginApiTest {

    private static final String BASE_URI = "https://api.example.com";
    private static String authToken;

    @BeforeClass
    public void setUp() {
        RestAssured.baseURI = BASE_URI;
    }

    @Test
    public void validLogin_ShouldReturn200AndToken() {
        String requestBody = "{ \"username\": \"validUser\", \"password\": \"validPass\" }";

        Response response = given()
            .contentType(ContentType.JSON)
            .body(requestBody)
            .when()
            .post("/login")
            .then()
            .assertThat()
            .statusCode(200)
            .body("token", notNullValue())
            .body("userId", notNullValue())
            .extract()
            .response();

        authToken = response.jsonPath().getString("token");
    }

    @Test(dependsOnMethods = "validLogin_ShouldReturn200AndToken")
    public void logout_ShouldInvalidateToken() {
        given()
            .header("Authorization", "Bearer " + authToken)
            .when()
```



```

        .post("/logout")
        .then()
        .assertThat()
        .statusCode(200)
        .body("message", equalTo("Logged out successfully"));
    }

    @Test
    public void invalidLogin_ShouldReturn401() {
        String requestBody = "{ \"username\": \"invalidUser\", \"password\": \"wrongPass\" }";

        given()
            .contentType(ContentType.JSON)
            .body(requestBody)
            .when()
            .post("/login")
            .then()
            .assertThat()
            .statusCode(401)
            .body("error", equalTo("Invalid credentials"));
    }
},

```

"reasoning": "We need to output Java code only, no markdown, no explanations. Should include imports, class, methods, beforeclass. Use RestAssured, TestNG, Maven style. Reusable base URI. Include request body creation (maybe using a Map or JSONObject). Include assertions for status code and important fields (e.g., token, message). Provide three test methods.

Make sure code compiles. Use static imports for RestAssured.given and Matchers.* maybe. Provide base URI in @BeforeClass. Also maybe store token for logout.

Let's produce code:

```

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

public class LoginApiTest {
    private static final String BASE_URI = "https://api.example.com";
    private static String authToken;

```

```

@BeforeClass
public void setUp() {
    RestAssured.baseURI = BASE_URI;
}

@Test
public void validLogin_ShouldReturn200AndToken() {
    String requestBody = "{ \"username\\\": \"validUser\\\", \"password\\\": \"validPass\\\" }";

    Response response =
        given()
            .contentType(ContentType.JSON)
            .body(requestBody)
        .when()
            .post("/login")
        .then()
            .assertThat()
                .statusCode(200)
                .body("token", notNullValue())
                .body("userId", notNullValue())
            .extract()
                .response();

    authToken = response.jsonPath().getString("token");
}

@Test(dependsOnMethods = "validLogin_ShouldReturn200AndToken")
public void logout_ShouldInvalidateToken() {
    given()
        .header("Authorization", "Bearer " + authToken)
    .when()
        .post("/logout")
    .then()
        .assertThat()
            .statusCode(200)
            .body("message", equalTo("Logged out successfully"));
}

@Test
public void invalidLogin_ShouldReturn401() {
    String requestBody = "{ \"username\\\": \"invalidUser\\\", \"password\\\": \"wrongPass\\\" }";

    given()

```

```

        .contentType(ContentType.JSON)
        .body(requestBody)
    .when()
        .post("/login")
    .then()
        .assertThat()
            .statusCode(401)
            .body("error", equalTo("Invalid credentials"));
    }
}

```

Make sure imports include json etc. That's fine. Provide code only.

Postman Screenshot:

The screenshot shows a Postman REST client interface. The request is a POST to `https://api.groq.com/openai/v1/chat/completions`. The body is raw JSON:

```

{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I - INSTRUCTION\n\nGenerate Rest Assured Java code for a Login API.\n\nMANDATORY RULES:\n\n1. Generate ONLY Java Rest Assured code.\n2. Do NOT include explanations, markdown, or extra text.\n3. Use TestNG as the test runner.\n4. Use meaningful class, method, and variable names.\n5. Include required imports.\n6. Use reusable base URI setup.\n7. Add assertions for
  ]
}

```

The response is a 200 OK status with a JSON body:

```

{
  "id": "chatcmpl-f1222cde-38de-414a-b56c-d8207f6f3949",
  "object": "chat.completion",
  "created": 1781712297,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport static io.restassured.RestAssured.*;\nimport static org.hamcrest.Matchers.*;\n\npublic class LoginApiTest {\n\n    private
      ]
    ]
  }
}

```

At the bottom left, it says "Import Complete".

S09-TextFromImage-base64(Classroom task)

Image to Base64 screenshot:

Image to Base64

Convert image to Base64 online and use the result string as data URI, img src, CSS background-url, and others. Sometimes you have to send or output an image within a text document (for example, HTML, CSS, JSON, XML), but you cannot do this because binary characters will damage the syntax of the text document. To prevent this, for example, you can encode image to Base64 and embed it using the data URI. Please note that the image to Base64 encoder accepts any images types with a size of up to 50 MB. If you are looking for the reverse process, check [Base64 to Image](#).

Datatype
Local File

Local File*
Choose file | No file chosen
Choose a file or drag and drop it here

Output Format
Plain text -- just the Base64 value

Encode image to Base64

Base64

```
/9j/4QC8RXhpZgAASUkqAAgAAAAABIBAwABAAAAQAAABBBQABAAAVyAAABBBQABAAAXgAAACgBAwABAAAAgAAABMCAwABAAAAQAAAGhBAABAAAAZgAAAAAAIAAAQAAAEgAAABAAABAAgBAkACABAAADAYITABKQcABAAAAAECAwAAACABAAADAYIDABAAAMQAAP//AAACoQAQAAAHgFAADoQAQAAQAAKOGAAAAAA/9sAQwAGBwYGBwYHhYKAgMgSkCwCPDgPwQgBwEBEQERAWIHUZFUFZS1eJ84cHIQeN1omJio2PjQyND5MRERX1pFFHym/8AAEQgGrQVAAwEIAIRAQMRAF/EABwAAIDAAABAAAAAABAgMEBQYHCP/EAFsQAAIBAgQDBQOFBwYjGgUCBwABAgMRBCExQQUQUYTYXBBYKRORQyscHRFSNcUnkY8DM2YpKT4RYKHENTVHSC8SUmNURVY3ODorMXZJ5JwT3FZKMER1bD4v/EABwBAQEBAQEBAQAAAAAAAAABAgMEBQc/xAA7EQEBAABAgQEBAUDBA1CAgIAAQIRIQhXEkFRcQqIMeTM3KBQpGhscEjUmIFFE0C0eFTkYDxBNz/9oADAMBAIRAxEAPwDswB8bnJewIQgQDAB3ANDQBj+4AwHFDc6F2JBoK3mC+8B32GtF42FcCBhmCJLYDC+uXBQwmb1Z6LwAGedwGQDAFoC12BfImPKDYYKheASa
```

The result of Base64 encoding will appear here

How to convert image to Base64 online

Postman base64_img variable:

GenAI-June

Overview Authorization Scripts **Variables** Runs

Variable	Value
baseUri	
base64_img	/9j/4QC8RXhpZgAASUkqAAgAAAAABIBAwABAAAAQAAABBBQABAAAVyAAABBBQABAAAXgAAACgBAwABAAAAgAAABMCAwABAAAAQAAAGhBAABAAAAZgAAAAAAIAAAQAAAEgAAABAAABAAgBAkACABAAADAYITABKQcABAAAAAECAwAAACABAAADAYIDABAAAMQAAP//AAACoQAQAAAHgFAADoQAQAAQAAKOGAAAAAA/9sAQwAGBwYGBwYHhYKAgMgSkCwCPDgPwQgBwEBEQERAWIHUZFUFZS1eJ84cHIQeN1omJio2PjQyND5MRERX1pFFHym/8AAEQgGrQVAAwEIAIRAQMRAF/EABwAAIDAAABAAAAAABAgMEBQYHCP/EAFsQAAIBAgQDBQOFBwYjGgUCBwABAgMRBCExQQUQUYTYXBBYKRORQyscHRFSNcUnkY8DM2YpKT4RYKHENTVHSC8SUmNURVY3ODorMXZJ5JwT3FZKMER1bD4v/EABwBAQEBAQEBAQAAAAAAAAABAgMEBQc/xAA7EQEBAABAgQEBAUDBA1CAgIAAQIRIQhXEkFRcQqIMeTM3KBQpGhscEjUmIFFE0C0eFTkYDxBNz/9oADAMBAIRAxEAPwDswB8bnJewIQgQDAB3ANDQBj+4AwHFDc6F2JBoK3mC+8B32GtF42FcCBhmCJLYDC+uXBQwmb1Z6LwAGedwGQDAFoC12BfImPKDYYKheASa
Add variable	

Request Body:

```
{
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "This is a image. Extract and return only the text exactly as shown,
preserving case."
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "data:image/png;base64,{{base64_img}}"
          }
        }
      ]
    }
  ]
}
```

Response Body:

```
{
  "id": "chatcmpl-f2ac17b4-fe43-499d-9252-d8f075076288",
  "object": "chat.completion",
  "created": 1781713743,
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "DIFFICULT \nROADS \nLEAD TO \nBEAUTIFUL \nDESTINATIONS"
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
```

```
    "queue_time": 0.256889682,  
    "prompt_tokens": 2497,  
    "prompt_time": 0.108676708,  
    "completion_tokens": 20,  
    "completion_time": 0.045258951,  
    "total_tokens": 2517,  
    "total_time": 0.153935659  
  },  
  "usage_breakdown": null,  
  "system_fingerprint": "fp_79da0e0073",  
  "x_groq": {  
    "id": "req_01kvb6maz1fqpa9q3l826nv7px",  
    "seed": 1868212212  
  },  
  "service_tier": "on_demand"  
}
```

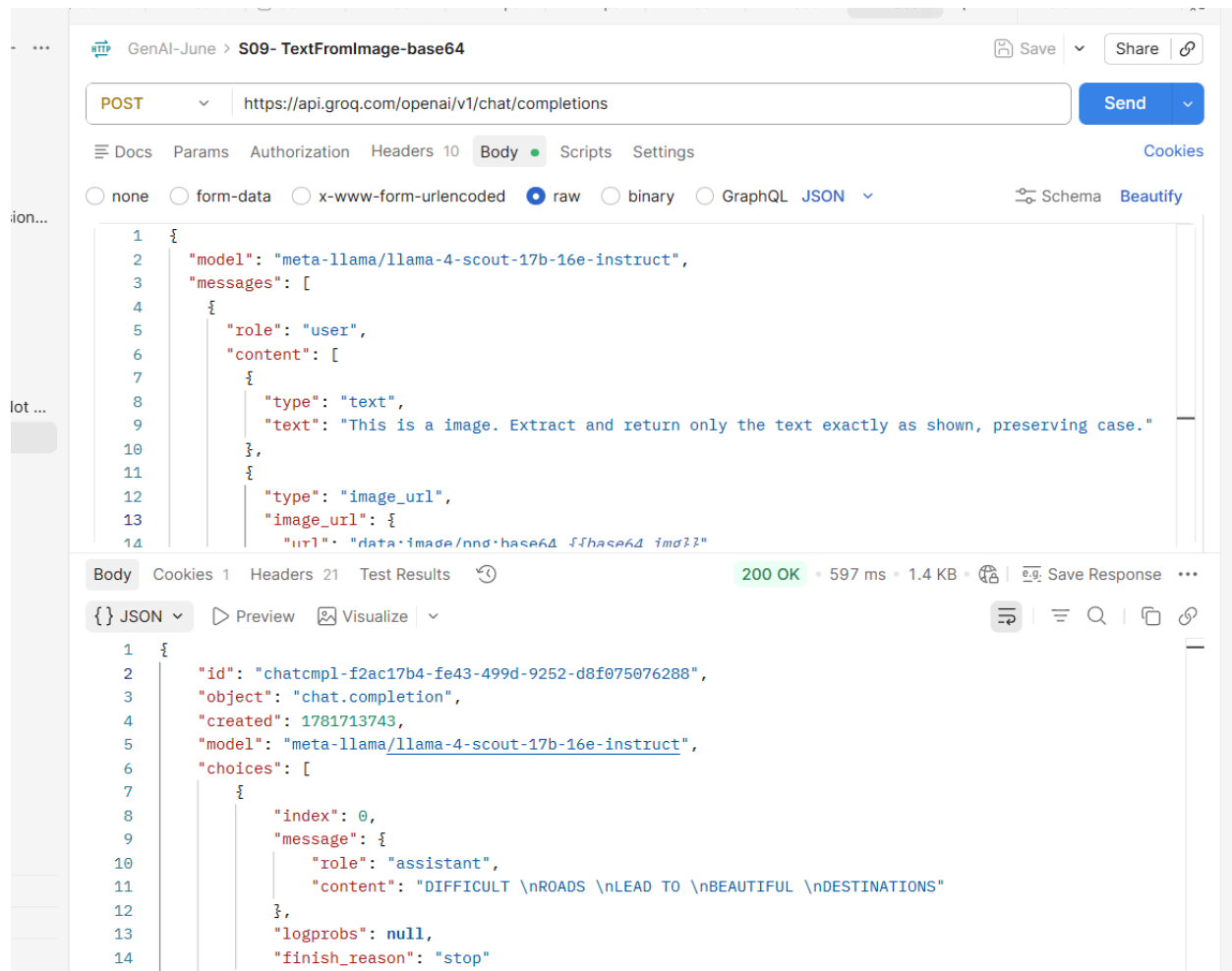
JSON to Plain Text output:

DIFFICULT
ROADS
LEAD TO
BEAUTIFUL
DESTINATIONS

Image uploaded initially:



Postman Screenshot:



Import Complete